

EXPERIMENT-30 Backward Chaining

Aim

To implement **Backward Chaining** in Prolog.

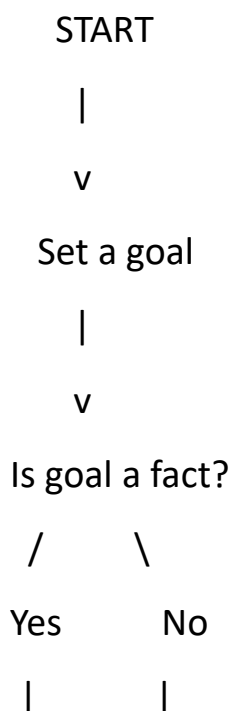
Objectives

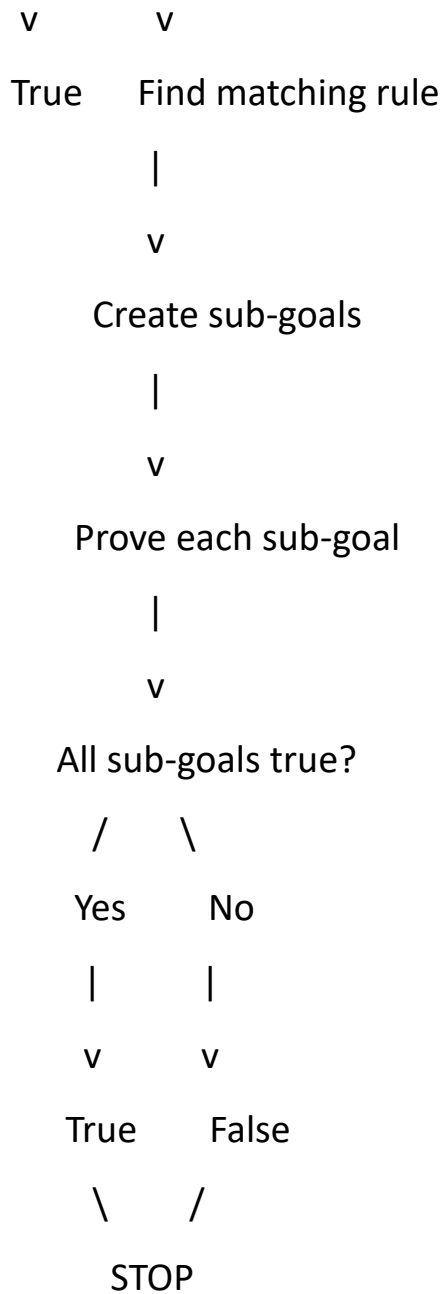
- To understand goal-driven reasoning.
- To prove a goal using available facts and rules.
- To understand how Prolog naturally performs backward reasoning.

Algorithm

1. Start with a goal.
2. Check whether the goal is a known fact.
3. If not, find a rule whose conclusion matches the goal.
4. Treat the rule's conditions as sub-goals.
5. Recursively prove all sub-goals.
6. If all sub-goals are true, the original goal is true.

Flowchart





Program

% Backward Chaining

fact(fever).

fact(cough).

rule(flu, fever).

```
rule(flu, cough).
```

```
rule(rest, flu).
```

```
prove(Goal) :-  
    fact(Goal).
```

```
prove(Goal) :-  
    rule(Goal, Condition),  
    prove(Condition).
```

Queries

```
?- prove(fever).
```

Output:

```
true.
```

```
?- prove(flu).
```

Output:

```
true.
```

```
?- prove(rest).
```

Output:

```
true.
```

```
?- prove(pneumonia).
```

Output:

HelloWorld.pl X		IO AI Agent
<pre>1 % Backward Chaining 2 3 % Facts 4 fact(fever). 5 fact(cough). 6 7 % Rules 8 rule(flu, fever). 9 rule(flu, cough). 10 rule(rest, flu). 11 12 % Prove a goal using backward chaining 13 prove(goal) :- 14 fact(goal). 15 16 prove(goal) :- 17 rule(goal, Condition), 18 prove(Condition). 19 20 % Main program 21 main :- 22 write('Backward Chaining Results'), nl, 23 write('-----'), nl, 24 25 write('prove(fever): '), 26 (prove(fever) => 27 write('true') 28 ; 29 write('false'))</pre>		<pre>STDIN Input for the program (optional) Output: Backward Chaining Results ----- prove(fever): true prove(flu): true prove(rest): true prove(pneumonia): false</pre>

Result

Backward chaining was successfully implemented by starting with a goal and recursively proving its conditions.

Conclusion

Prolog naturally supports backward chaining because it starts with a query and searches for facts and rules that can prove that query.